# INTERACTIVE SALES ANALYTICS DASHBOARD

*Technical Report*

Data Science & Analytics Internship Task

Technologies: MySQL | Node.js | Express | React | Vite | Recharts

# 1. Introduction

The Interactive Sales Analytics Dashboard is a full-stack data analytics application developed to transform transactional sales data into meaningful business insights.

The project follows a practical business intelligence workflow. Sales data is stored in a MySQL database, retrieved and analyzed through a Node.js and Express backend, and presented through an interactive React single-page application.

The dashboard provides an overview of sales performance through key performance indicators, revenue trends, product-level analysis, regional performance, and interactive filtering.

# 2. Objectives

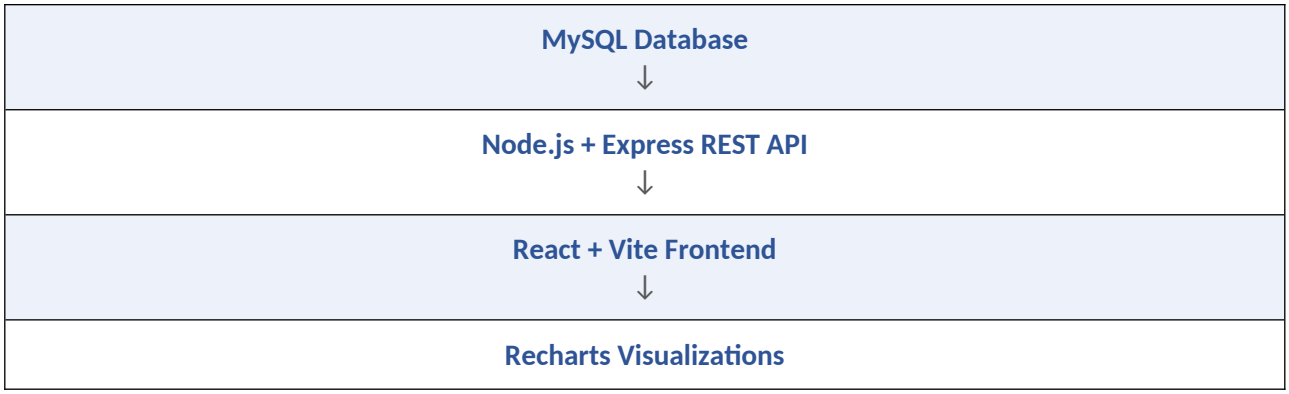
- Extract sales data from a MySQL relational database.
- Validate the available data and perform appropriate data-quality checks.
- Transform transactional records into useful analytical metrics.
- Develop REST API endpoints using Node.js and Express.
- Perform exploratory analysis using SQL aggregation and backend processing.
- Visualize sales trends and comparisons using React and Recharts.
- Implement interactive Region and Category filters.
- Develop a responsive and user-friendly analytics dashboard.

# 3. Technology Stack

| Technology | Role in the Project |
|---|---|
| MySQL | Relational database for storing transactional sales records |
| Node.js | Backend runtime environment for data processing and API development |
| Express | REST API endpoints that provide sales and analytical data to the frontend |
| mysql2/promise | Asynchronous communication between Node.js and MySQL |
| dotenv | Manages database configuration through environment variables |
| React | Builds the single-page dashboard interface |
| Vite | Frontend development and production build tool |
| Recharts | Creates revenue trend, product, and regional visualizations |
| CORS | Allows communication between the React frontend and Node.js backend locally |

## 4. System Architecture

The application follows a simple layered architecture:




The MySQL database acts as the source of transactional sales data. The Node.js backend retrieves the records and performs aggregation and analytical calculations. The React frontend consumes the API responses and presents the results through KPI cards, charts, and interactive filters.

The separation between the database, backend, and frontend makes the application easier to maintain and extend.

## 5. Data Extraction and Cleaning

The main database table used by the application is named "sales". The table contains the following fields:

- id
- order_date
- product_name
- category
- region
- quantity
- unit_price

The "id" field is the primary key and is automatically generated. The remaining application fields are defined as NOT NULL in the database schema.

**Revenue Calculation**

```
Revenue = quantity × unit_price
```

Data-quality checks were performed for missing values, duplicate identifiers, invalid quantities, invalid prices, dates, categories, and regions.

The dataset did not contain significant data-quality issues requiring destructive cleaning. Therefore, unnecessary modifications to the original transactional data were avoided.

The project focuses on appropriate validation and transformation rather than artificially modifying already-valid records.

## 6. Exploratory Data Analysis

Exploratory analysis was performed using SQL aggregation queries exposed through the Node.js backend. The main analytical calculations include:

| Metric | Calculation |
|---|---|
| Total Revenue | SUM(quantity × unit_price) |
| Total Orders | COUNT(*) |
| Units Sold | SUM(quantity) |
| Average Sale Value | AVG(quantity × unit_price) |
| Revenue Trends | Monthly revenue grouped by year and month |
| Top Products | Products ranked according to total revenue |
| Regional Performance | Regions ranked according to total revenue |

# 7. Analysis Results

The dataset contained 30 sales records.

| Overall Metric | Value |
|---|---|
| Total Records / Orders | 30 |
| Total Units Sold | 444 |
| Total Revenue | $77,655.00 |
| Average Sale Value | $2,588.50 |

## 7.1 Category Analysis

| Category | Units Sold | Revenue |
|---|---|---|
| Electronics | 99 | $62,940.00 |
| Accessories | 345 | $14,715.00 |

Electronics generated significantly more revenue despite having fewer units sold. This indicates that electronics had a substantially higher average transaction value compared with accessories.

The category results demonstrate why both unit volume and revenue are useful metrics when evaluating sales performance.

## 7.2 Regional Analysis

| Region | Units Sold | Revenue |
|---|---|---|
| East | 99 | $22,715.00 |
| South | 118 | $20,510.00 |
| North | 122 | $19,615.00 |
| West | 105 | $14,815.00 |

East was the highest-performing region by revenue, generating $22,715.00.

North sold the highest number of units with 122 units, but it did not generate the highest revenue. This demonstrates that sales volume and revenue do not necessarily identify the same top-performing region.

# 8. Dashboard Functionality

The React dashboard provides an interactive overview of sales performance.

### KPI Cards

- Total Revenue
- Total Orders
- Units Sold
- Top Region

### Visualizations

- Revenue Trends — a line chart showing revenue by month
- Top Products — a bar chart ranking products according to revenue
- Regional Performance — a bar chart comparing revenue across regions

### Interactivity

The dashboard also provides interactive Region and Category dropdown filters. When a filter is selected, the dashboard requests filtered KPI data through the "/api/dashboard" endpoint. The selected filters are also applied to the sales records used by the visualizations.

Selecting "All Regions" or "All Categories" removes the corresponding filter and restores the complete dataset.

The interface also includes loading and error states and has been designed to remain usable across different screen sizes.

## 9. API Endpoints

| Endpoint | Description |
|---|---|
| GET / | Basic API health message confirming the backend is running |
| GET /api/sales | Sales records including calculated revenue |
| GET /api/summary | Overall revenue, order count, and units sold |
| GET /api/revenue-trends | Monthly revenue aggregates for the revenue trend chart |
| GET /api/top-products | Products ranked by revenue |
| GET /api/regional-performance | Regional revenue and units sold |
| GET /api/dashboard | Filtered dashboard summary metrics |

### Example Filtered Requests

```
/api/dashboard?region=East
/api/dashboard?category=Electronics
/api/dashboard?region=East&category=Electronics
```

## 10. Project Structure

The project is divided into separate backend and frontend components.

**backend/**

```
.env.example
db.js
server.js
test-db.js
package.json
package-lock.json
```

**frontend/**

```
src/
  App.jsx
  App.css
  index.css
  main.jsx
public/
package.json
vite.config.js
package-lock.json
```

Environment-specific credentials are stored locally in ".env" and are excluded from version control.

## 11. Challenges Faced

Several practical challenges were encountered during development.

- Configuring the local MySQL environment and establishing a successful connection between MySQL and Node.js.
- Translating transactional sales records into useful analytical metrics using SQL aggregation functions such as SUM, COUNT, AVG, GROUP BY, and ORDER BY.
- Connecting multiple backend API endpoints to the React frontend, which required handling asynchronous requests and different data states.
- Implementing optional Region and Category filters, which required parameterized queries so filters could be applied without constructing unsafe SQL statements.
- Adjusting the dashboard layout and Recharts visualizations to maintain readability and responsiveness across different screen sizes.

## 12. Testing and Verification

Database connectivity was verified using the Node.js database test script.

The backend API was tested through its local endpoints to confirm that MySQL data could be retrieved successfully.

The React dashboard was tested with different filter combinations, including:

- All Regions + All Categories
- East + All Categories
- All Regions + Electronics
- East + Electronics

The frontend production build was also tested using the Vite build command.

The dashboard successfully displayed the expected sales metrics and visualizations.

## 13. How to Run the Project

**Prerequisites**

- Node.js and npm
- MySQL Server
- A MySQL database containing the "sales" table

**Backend Setup**

```
1. Open a terminal in the backend directory.
2. Run npm install.
3. Create a .env file using .env.example.
4. Enter the local MySQL connection details.
5. Run node test-db.js to verify database connectivity.
6. Start the backend using node server.js.
```

**Frontend Setup**

```
1. Open a second terminal in the frontend directory.
2. Run npm install.
3. Run npm run dev.
4. Open the Vite URL displayed in the terminal.
```

The backend runs on port 5000 and the frontend normally runs on port 5173.

## 14. Conclusion

The Interactive Sales Analytics Dashboard demonstrates a complete data analytics workflow from relational data storage to interactive reporting.

The project successfully integrates MySQL, Node.js, Express, React, and Recharts into a single application. Transactional sales data is transformed into meaningful metrics such as total revenue, units sold, average sale value, product performance, and regional performance.

The interactive filters allow users to explore the data dynamically instead of viewing only static results.

The project provided practical experience with SQL querying, data aggregation, backend API development, React-based visualization, responsive dashboard design, and the overall workflow involved in building a small business intelligence application.